



First semester 2022/2023
Advanced topics in software development
Homework Assignment # awxz3cdfv3e
Event Processing System Refactoring
ETA: 12 Dec midnight.

Create by : Ahmad istatieh 12217056

Under supervision

Dr. Mustafa Assaf
En.Mohammad Qzeh

Before the Refactor How the System Works Now

We have a system called Event Processing System.

This system is responsible for processing different types of events such as user actions , system related events , and security events.

Let's imagine a scenario where an event occurs in the system.

When an event is created, it contains a type "such as USER, SYSTEM, or SECURITY" , along with a payload that represents the event's data , The event may also contain several flags that indicate whether the payload should be encrypted , compressed , or enriched with additional metadata.

Once the event is ready, it is sent directly to the EventProcessor, which is the central component responsible for handling the entire event processing workflow , The EventProcessor starts by validating the event. It checks whether the payload exists and is not empty , If the payload is invalid, the event processing stops immediately.

If the event is valid, the EventProcessor then proceeds to notify other subsystems. It updates the system dashboard by sending the event to the Dashboard component , and it logs the event information using the Logger component.

After that, the EventProcessor performs multiple transformations on the event payload. Based on several boolean flags inside the Event object, the processor decides whether the payload should be encrypted, compressed, or combined with metadata. These transformations are executed sequentially using conditional statements inside the EventProcessor itself.

Once the payload transformation is complete, the EventProcessor generates a unique identifier for the event based on the current time and the processed payload. This ID is then stored inside the event object.

The processed data, along with the generated event ID, is then persisted by calling the Database component directly. The Database saves the event data immediately without using any abstraction or reuse mechanism.

Finally, the EventProcessor performs additional behavior based on the event type. It uses a series of conditional if/else statements to determine what action should be taken for each event type , for security events, the processor performs extra analysis and directly triggers a security alert , for user events, it executes user-specific logic , for system events, it logs system-related audit information.

All of these responsibilities including validation , transformation , notification , persistence , and type specific behavior are handled within the same EventProcessor class.

Key Problems in This Code :

Issue 1: Centralized Event Processing (Single Responsibility Violation) :

The main issue in the current implementation is the `EventProcessor.process()` method. This method handles all stages of event processing instead of delegating responsibilities to specialized components.

Specifically, the method performs: Event validation , Dashboard updates , Logging , Payload transformations , Event ID generation , Database persistence , Event type specific behavior

All of these operations are implemented inside a single method , which causes the logic to be tightly coupled and difficult to maintain. Any change in one processing step requires modifying the same method increasing the risk of errors as the system evolves.

can be seen in the following code :

```
public void process(Event e) {
    // validate the new event!
    if (e.getPayload() == null || e.getPayload().isEmpty()) {
        System.out.println("Invalid event");
        return;
    }

    // notify various subsystems with new event.
    dashboard.updateMetrics(e);
    logger.log(e);

    // transform (lots of if/else)
    String data = e.getPayload();
    if (e.isEncrypt()) {
        // naive encryption
        data = "ENC(" + data + ")";
    }
    if (e.isCompress()) {
        data = "CMP(" + data + ")";
    }
    if (e.isAddMetadata()) {
        data = "META(" + e.getMetadata() + ")::" + data;
    }

    // persist
    e.setId(System.currentTimeMillis() + "-" + Math.abs(data.hashCode()));
    database.save(e.getId(), data);

    // extra behavior depending on type
    if ("SECURITY".equals(e.getType())) {
        // duplicate security handling across code
        System.out.println("[SECURITY] extra analysis for " + e.getId());
        // directly send to security monitor (hard-coded)
        System.out.println("[SecurityMonitor] alert for " + e.getId());
    } else if ("USER".equals(e.getType())) {
        System.out.println("[USER] user-specific step for " + e.getId());
    } else if ("SYSTEM".equals(e.getType())) {
        System.out.println("[SYSTEM] system audit log " + e.getId());
    }
}
```

Because all responsibilities are centralized in one place, the system is hard to extend , difficult to test , and not well prepared for future changes such as adding new event types or processing steps.

Issue 2: Event Type Handling with String and if/else (Poor Extensibility)

The current implementation handles event types using plain strings combined with a chain of if/else conditions inside EventProcessor.

The event type is stored as a String in the Event class :

```
private String type; // "USER","SYSTEM","SECURITY"
```

and is checked like this :

```
// extra behavior depending on type
if ("SECURITY".equals(e.getType())) {
    // duplicate security handling across code
    System.out.println("[SECURITY] extra analysis for " + e.getId());
    // directly send to security monitor (hard-coded)
    System.out.println("[SecurityMonitor] alert for " + e.getId());
} else if ("USER".equals(e.getType())) {
    System.out.println("[USER] user-specific step for " + e.getId());
} else if ("SYSTEM".equals(e.getType())) {
    System.out.println("[SYSTEM] system audit log " + e.getId());
}
```

This design cause two main problems:

Error prone type representation

Using raw strings for event types makes the code fragile , Any typo in the string will not be detected at compile time and can break the logic silently.

Hard to extend (Disagree Open/Closed Principle)

Adding a new event type requires editing the process() method and adding another else if branch , This means the core processing logic must be modified every time a new type is introduced , making the system difficult to extend and maintain.

Because of this design , the event type handling is tightly coupled to the EventProcessor implementation and does not scale well as the number of event types grows.

Issue 3: Too Many Flags Inside Event (Feature Flags Controlling Logic Externally)

The Event class contains several boolean flags that control how the payload should be processed:

```
private boolean encrypt;  
private boolean compress;  
private boolean addMetadata;  
private String metadata;
```

These flags are then used inside `EventProcessor.process()` to decide which transformations to apply:

```
// transform (lots of if/else)  
String data = e.getPayload();  
if (e.isEncrypt()) {  
    // naive encryption  
    data = "ENC(" + data + ")";  
}  
if (e.isCompress()) {  
    data = "CMP(" + data + ")";  
}  
if (e.isAddMetadata()) {  
    data = "META(" + e.getMetadata() + "):" + data;  
}
```

This design introduces the following problems:

Growing complexity with every new feature

Each new processing option will require :

A new flag in the Event class.

New if conditions in EventProcessor.

Over time, this turns the processing code into a long list of conditionals that are hard to manage.

Mixing data with processing decisions

The Event object is supposed to represent the event data (type, payload, id) , but it also stores processing decisions (encrypt, compress, addMetadata) , This blurs the separation between what the event is and how it should be processed, making both the Event and EventProcessor more complicated.

Because of these flags, the processing logic becomes scattered and harder to extend , Adding or changing transformations always requires modifying the core processing method and the Event structure at the same time.

Issue 4: Tight Coupling to Concrete Components (Database, Dashboard, Logger, Security Logic)

The EventProcessor directly depends on concrete classes instead of abstractions :

```
private Database database;  
private Dashboard dashboard;  
private Logger logger;
```

It also contains hard coded security behavior :

```
// duplicate security handling across code  
System.out.println("[SECURITY] extra analysis for " + e.getId());  
// directly send to security monitor (hard-coded)  
System.out.println("[SecurityMonitor] alert for " + e.getId());
```

This causes two main problems:

Hard to change or replace dependencies

If the system later needs to:

Replace Database with another storage mechanism , Change how logging works , Or move security alerts to another module

then the EventProcessor itself must be modified , This increases coupling and makes the class sensitive to changes in other parts of the system.

Difficult to test and mock

Because EventProcessor is directly tied to real implementations, it is harder to unit test in isolation. You cannot easily swap in test doubles or mock components without changing the class.

This tight coupling reduces flexibility and makes the overall architecture less maintainable as the system evolves.

After the Refactor How the System Works Now

After the refactor, the Event Processing System no longer relies on a single, complex method that handles everything using conditional logic. Instead, the system now uses design patterns to make event processing clearer, more flexible, and easier to extend.

Let's return to our example. Ahmad triggers a new USER event by clicking on a button in the system.

This time, instead of manually deciding how the event should be handled, the system first goes to the EventProfileRegistry.

The registry already contains preconfigured event profiles for each event type (USER, SECURITY, SYSTEM).

Rather than rebuilding the event configuration every time , the system simply clones a predefined prototype profile and assigns it to the new event , This avoids repeated configuration logic and makes the system faster and more consistent.

Once the event profile is ready, the event is passed to the EventProcessor, which coordinates the execution flow.

The processor does not use if/else statements to distinguish between event types anymore. Instead, it selects the appropriate behavior using the Strategy Pattern, where each event type is handled by its own dedicated EventHandler.

Before storing the processed event, the system does not interact with the database directly. Instead, the request goes through the EventRepositoryProxy.

The proxy retrieves a connection from the Connection Pool, ensuring that existing database connections are reused instead of creating new ones for every event.

After the connection is acquired, the proxy forwards the request to the real repository, which stores the event data.

Once the operation is completed, the proxy releases the connection back into the pool, keeping resource usage efficient and controlled.

Finally, the selected event handler completes any additional event-specific actions, such as triggering a user action or a security alert.

Now, if Ahmad triggers another event, or if the system processes different types of events, the flow remains efficient and organized.

How Each Problem Was Solved :

Issue 1: Rebuilding Event Configuration Repeatedly (Solved with Prototype + Registry Pattern) :

In the refactored version, event configurations are no longer rebuilt from scratch. Instead the Prototype Pattern is used along with an EventProfileRegistry.

Each event type (USER, SECURITY, SYSTEM) has a predefined prototype profile that is created once and stored in the registry.

When a new event is processed, the system simply clones the corresponding profile, ensuring fast and consistent configuration.

This removes duplicated setup logic and makes event processing more efficient and predictable.

Example (fixed code):

```
EventProfile profile = profileRegistry.getProfile(event.getType());
```

Issue 2: No Real Connection Pool (Solved with Connection Pool Pattern) :

After refactoring, a Connection Pool was introduced.

The pool maintains a fixed number of reusable database connections that are acquired when needed and released back after use.

This approach prevents unnecessary connection creation , improves performance , and ensures controlled resource usage.

Example (fixed code):

```
@Override 2 usages new *
public void save(Event event, String data) {
    DbConnection connection = pool.acquire();
    try {
        System.out.println("Using DB connection #" + connection.getId());
        target.save(event, data);
    } finally {
        pool.release(connection);
    }
}
```

Issue 3: Direct Access to Database Logic (Solved with Proxy Pattern) :

In the original design , event processing logic interacted directly with the database layer , This created tight coupling and made it difficult to add additional control logic such as connection management , To resolve this, the Proxy Pattern was applied using EventRepositoryProxy.

The proxy acts as an intermediary between the processor and the real repository. It manages connection acquisition and release while delegating the actual storage operation to the real repository.

This adds a clean control layer without changing the core storage logic.

Example (fixed code):


```
EventRepository realRepo = new DatabaseEventRepository();  
EventRepository proxyRepo = new EventRepositoryProxy(realRepo, pool);
```

Issue 4: Long Conditional Logic for Event Types (Solved with Strategy Pattern) :

After refactoring , the Strategy Pattern was introduced , Each event type now has its own EventHandler implementation , and the correct handler is selected dynamically at runtime.

This removes conditional logic and allows new event types to be added without modifying existing processing logic.

Example (fixed code):

```
EventHandler handler = handlers.get(event.getType());  
if (handler != null) {  
    handler.handle(event);  
}
```

Design Decisions

While working on refactoring the Event Processing System, the main goal was to clean up the code and make it easier to understand and extend in the future.

Overall Design Approach

In the original version , most of the work was done inside one class.

That class was responsible for processing events, deciding how each event should behave , and dealing with the database , Because everything was mixed together, the code became hard to read and risky to change.

To improve this , the logic was divided into smaller , focused classes.

Each part of the system now has a clear responsibility , which makes the code easier to follow and simpler to modify later if needed.

Using Prototype Pattern

Event configurations used to be decided manually every time an event was processed.

Instead of repeating the same setup logic, predefined event profiles are now created once and stored in a registry.

When a new event arrives, the system simply clones the related profile instead of creating it from scratch , This made the configuration process cleaner and more consistent.

Using Strategy Pattern

Different event types require different behavior.

Handling this using if/else statements would quickly make the code messy as more event types are added.

To avoid this , each event type has its own handler , The system selects the correct handler at runtime, keeping the processing logic simple and organized.

Using Proxy and Connection Pool Patterns

Direct interaction with the database was avoided to keep the processing logic clean.

A proxy was added to control access to the database and handle connection management.

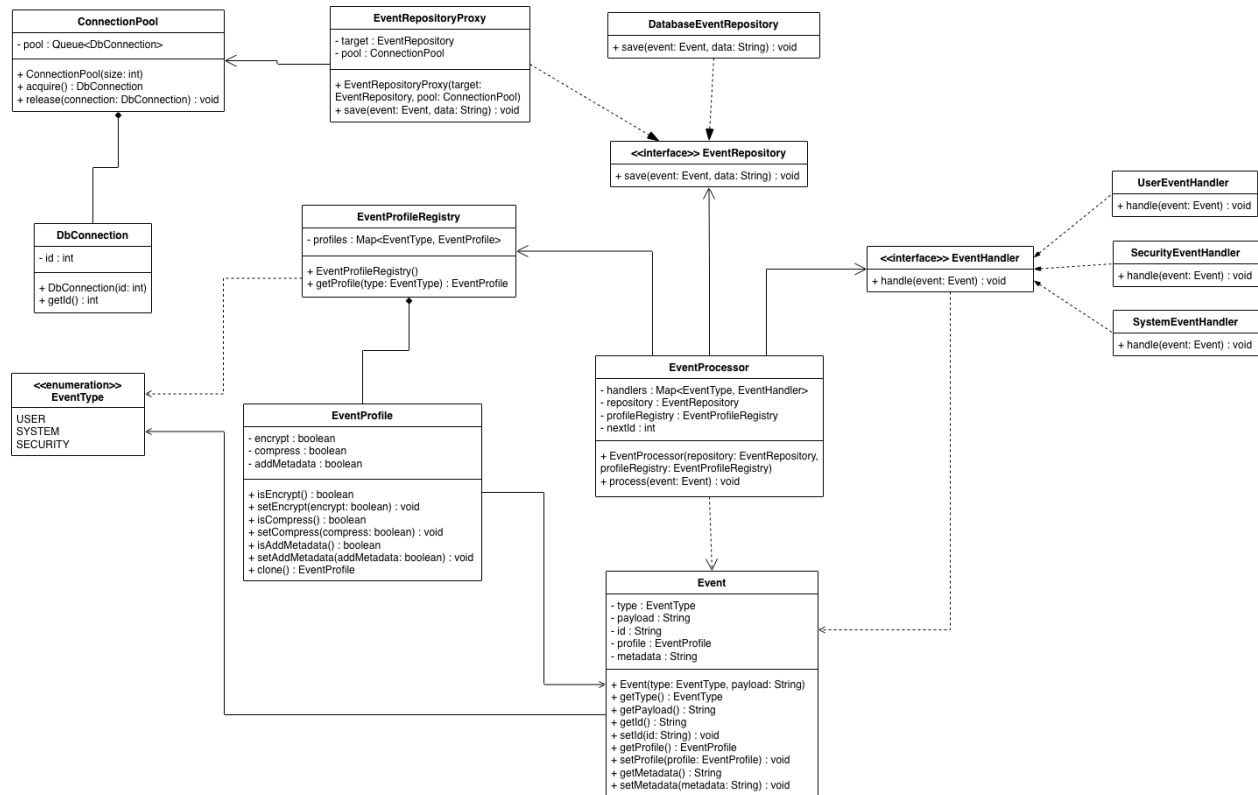
A connection pool was also introduced so database connections can be reused instead of created every time , This approach makes the system more efficient and easier to control.

these design decisions make the system Easier to read , Easier to extend , Better organized , More efficient in handling resources.

The output of the Event Processing System :

```
Using DB connection #0
Event[USER] id=1
stored with data: ENC(user-click)
USER handler done for 1
=====
Using DB connection #1
Event[SECURITY] id=2
stored with data: CMP(failed-login)
SECURITY handler done for 2
```

The UML :



In the end : refactoring the Event Processing System improved its structure and flexibility , Using well known design patterns helped separate responsibilities and made the system easier to maintain and extend in the future.